// BLOG

Technical Overview of Akka.Cluster.Sharding in Akka.NET

Shares

How Akka.Cluster.Sharding Allocates Shards, Rebalances, and More

By Bartosz Sypytkowski · January 31, 2017 · 11 minutes to read

AKKA.NET CLUSTERING

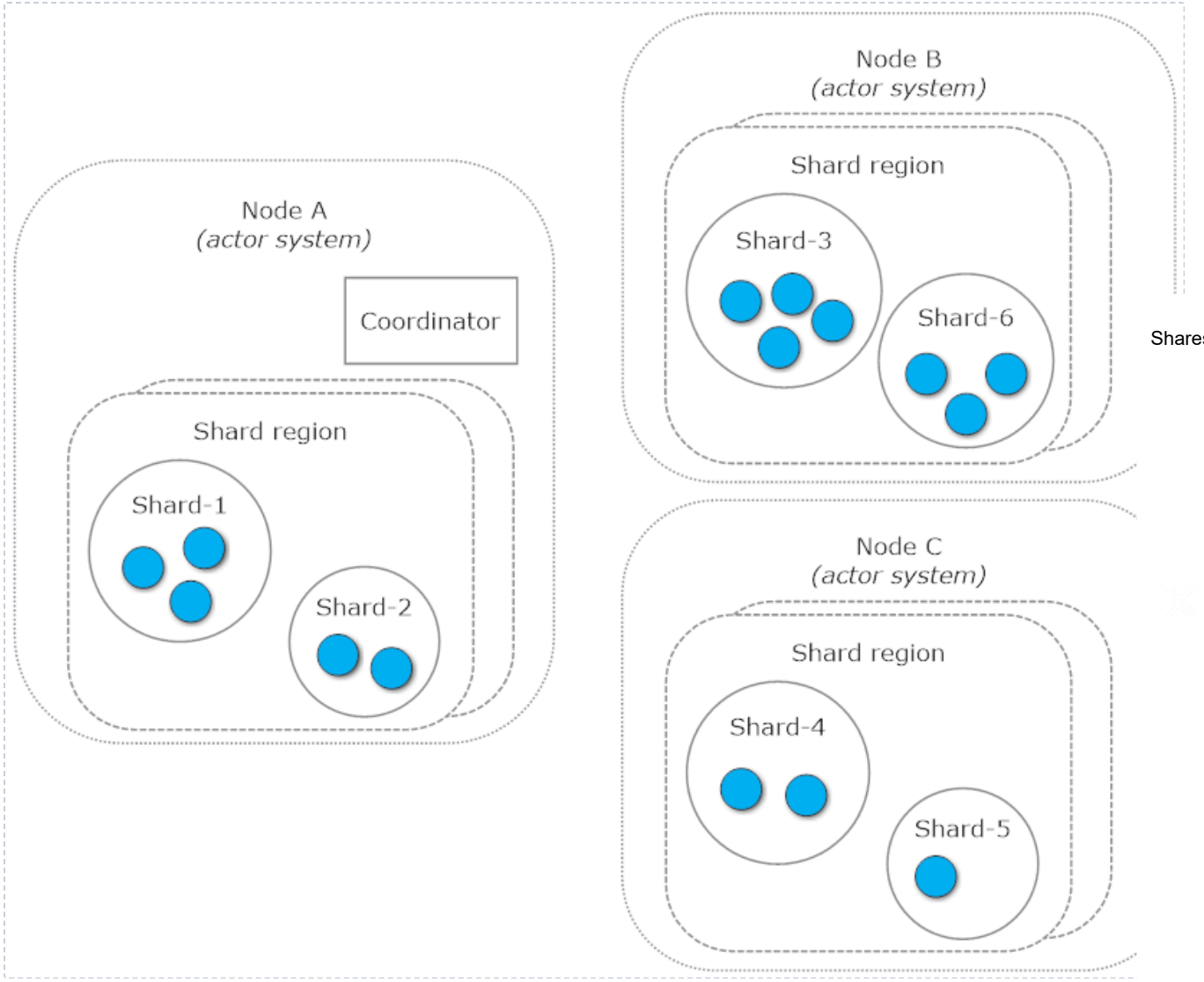
> *We have an updated guide to Akka.Cluster.Sharding that incorporates newer APIs and practices. Please see "Distributing State Reliably with Akka.Cluster.Sharding" instead.*

In our previous post about using Akka.Cluster.Sharding we looked at the module from an end-user's perspective. Today we'll provide a little more insights into how this plugin works internally.

Cluster sharding depends on several types of actors:

- **Coordinator** - one per entity type for an entire cluster.
- **Shard region** - one per entity type per each cluster node, where sharding should be enabled.
- **Shard** - there can be many on each shard region, and they can move between shard regions located on different nodes.
- **Entity** - actors defined by the end-user. There can be many on each shard, but they are always bound to a specific shard.

All of them can be visualized using diagram below:



Shares

If you look at actor paths of the entities you've created you'll see that they reflect the structure of that hierarchy. They follow the pattern

petabridge light

are responsible for shard actor and entity actor.

When a shard region is used to send a message to an entity inside a shard living on a different node for a first time, it must resolve location of that shard first. A cluster-global register of every shard is tracked by the coordinator actor. Coordinators are singletons, which means, that there will never be more than one instance of it living inside the same cluster and it is usually placed on the oldest/seed node. It's a heuristic driven choice as oldest nodes usually have tendency to be a long-living ones in situations when we add or remove additional machines to scale cluster capacity to match incoming traffic.

Once a shard region receives an address location of the shard it will cache it (until a shard has to rebalance to another node.) All subsequent messages send to that shard

will happen without need of calling coordinator anymore, which avoids turning the coordinator into a bottleneck.

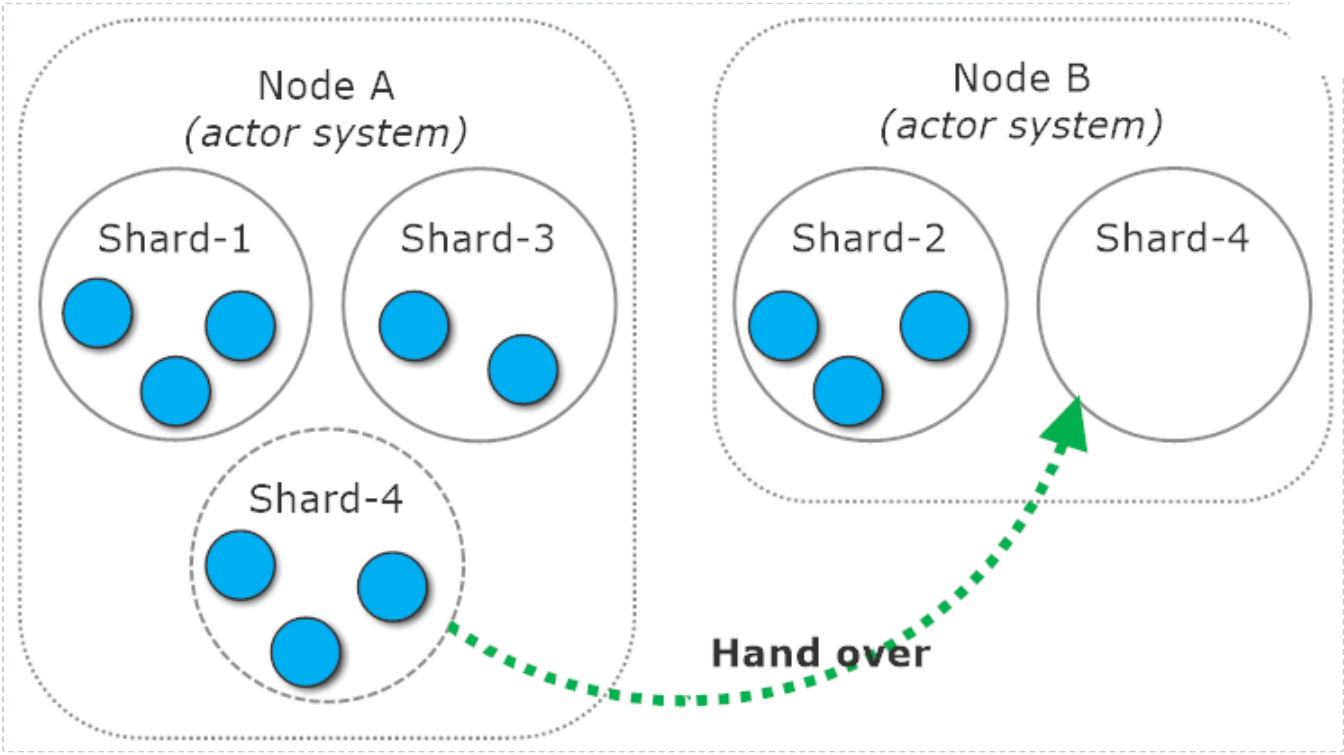
Rebalancing

Rebalancing is a process in which entities migrate from one node to another in order to more evenly distribute entities across the cluster. However migration process is expensive and making it one by one for every affected entity is expensive. For this reason, entities always live in shards and when we migrate them we always migrate the whole shard.

Shares

Shards also provide “islands” of locality that can be used to optimize message communication and reduce network usage.

Rebalancing can occur when node agreed to exchange shards with each other and one node is much more heavily loaded with shards than another. It can be triggered within given time interval (which can be set using `akka.cluster.sharding.rebalance-interval`), but most often you’ll see it happen once a new node joins the cluster.



Once shards are rebalanced or restarted on new node they **won’t** resurrect their entities to match the state from before handing shard over between nodes. An entity will have to be addressed via message in order to be started again. This may sometimes go in conflict with our application logic i.e. when entity has scheduled itself a message and it’s

expecting to receive it a next minute or two. Fortunately, this behavior can be changed by setting the `akka.cluster.sharding.remember-entities` flag on.

At this point you might ask: *what happens to all messages sent to a shard's entities during hand over?* Answer is simple: they are buffered until shard migration finishes. Once that happens all buffered messages are redirected to new shard location, causing entities to be resurrected if necessary. This protects us from accidentally loosing messages, increasing reliability of our system.

Shares

> *NOTE: keep in mind that in Akka.NET by default all messages are kept in memory. This also means that in case of unexpected machine crashes all unhandled messages will be lost. It's a conscious decision - we deliberately trade off reliability for high performance. If this is not the case for you, consider using durable queue or log (like RabbitMQ or Kafka) in front of your actor system or make your actors' states persistent by using an Akka.Persistence module or your own hand-rolled repository.*

Shard allocation and rebalance is conducted by the logic defined inside components inheriting from `IShardAllocationStrategy` interface. By default a `LeastShardAllocationStrategy` will be used; it simply allocates new shards on the least populated nodes in terms the number of shards, **not** the number of entities living on them and rebalances them there if a specified difference between the most and least populated nodes has been exceed. You can configure that threshold by using `akka.cluster.sharding.least-shard-allocation-strategy.rebalance-threshold` setting. You can also specify your own strategy when starting a new shard region on the node.

When a rebalancing is going to occur a shard will send a hand off message to all of it's entities. By default this message will be a `PoisonPill` causing them all to stop immediately but you can replace it with a custom one in case if you need to perform some preparations before migrating.

How to determine correct size of a shard?

There is no easy answer here. While having a large number of shards with a small number of entities are less expensive to migrate, the overall memory footprint and costs

of keeping their location up to date on each node increases along with the number of shards. With bigger shards this overhead is smaller, but cost of rebalancing is higher.

Ultimately,shard size should fit the type of allocation strategy used - because it affects how often rebalancing will occur. But with defaults, shard migration is usually pretty rare situation, therefore prefer to grow in size of each shard instead of increasing the number of shards.

Shares

Using Akka.Cluster.Sharding with Lighthouse

A lot of people have a one common problem when it comes to combining Akka.Cluster.Sharding with Lighthouse on default settings. We'll propose a solution, b first let us understand the nature of a problem.

Do you remember how we were talking about role of shard coordinator? As we said, coordinators by default will try to be established on the oldest nodes. What does that have to do with Lighthouse? Lighthouse is basically a stub of an Akka.NET `ActorSyst` with cluster capabilities. Since it's used as a seed, it also means that Lighthouse node are usually considered the oldest ones. However they have no idea on how to deal wi sharding protocol. Therefore, once cluster sharding nodes try to access a coordinato they'll start to look for it on the Lighthouse nodes where it cannot be found.

So how can we stop cluster sharding from trying to communicate with invalid nodes? Akka.NET allows us to build heterogeneous clusters. By using cluster roles, we can group individual nodes by their responsibilities. Akka.Cluster.Sharding module can also make use of them. By specifying `akka.cluster.sharding.role` we can limit a number of nodes, on which cluster sharding is expected to operate. By excluding lighthouse nodes from that role, we explicitly say cluster sharding to stop looking for coordinator there.

Communication with external entities

There are situations in which you may want to use automatic message routing provided by cluster shards from nodes where shards themselves are not going to be hosted. This is still possible by using shard region proxies:

```
var sharding = ClusterSharding.Get(system);
var proxy = sharding.StartProxy(
    typeName: "user",    // equal to shard region typeName, we
want to connect
    role: "user-role",  // role of nodes, on which we should look
for shard regions
    messageExtractor: new MyMessageExtractor());
```

Copy

Shares

Since proxies are not responsible for creating entities things like actor `Props` or allocation strategy are not needed here. However, you still need to provide message extractor as this component is responsible for message routing to correct destination

Getting shard metadata

While cluster sharding deals with a lot of actor management concerns for you, it's not complete black box. Shards and regions provide a variety of messages that can be us to query for their current state and retrieve all necessary metadata. Let's quickly enumerate some of them.

Query messages send to shard region:

- `GetClusterShardingStats` → `ClusterShardingStats` : a reply contains info about which nodes host which shards (and how many entities each of the shards host). Because this may be quite heavy operation, a supplied timeout parameter may be used to prevent from waiting for the response indefinitely.
- `GetShardRegionStats` → `ShardRegionStats` works like the one above, but it's limited only to the current node.
- `GetCurrentRegions` → `CurrentRegions` : a reply will return addresses of all known nodes hosting entities of the same type name, as the shard region you've send a query to.
- `GetShardRegionState` → `CurrentShardRegionState` : reply will contain identifiers of all entities (grouped by their shard IDs), that live currently on the asked shard region.

Query message, that can be send to a particular shards:

- `Shard.GetCurrentShardState` → `CurrentShardState` : reply contains a list of all entity IDs living currently in target shard.
- `Shard.GetShardStats` → `ShardStats` is more lightweight version, as it'll only return a number of hosted entities (within a scope of asked shard).

All of those can be used to explore current structure and localization of shards and entities, making it easier for you to monitor and navigate through them.

Shares

About the Author

Bartosz Sypytkowski is one of the Akka.NET core team members and contributor sinc 2014. He's one of the developers behind persistence, streams, cluster tools and sharc plugins, and a maintainer of Akka F# plugin. He's a great fan of distributed functional programming. You can read more about his work on his blog: http://bartoszsypytkowski.com

Phobos

Observe and Monitor Your Akka.NET Applications with Phobos

Phobos automatically instruments your Akka.NET applications with OpenTelemetry — traces, metrics, and logs with built-in dashboards.

Buy Now Learn More

// RELATED

May 13, 2026

Case Study: Luware Building Mission-Critical Voice Services with Akka.NET

CASE STUDIES AKKA.NET CLUSTER SHARDING

April 13, 2026

Managing Long-Running Tasks Inside Akka.NET Actors

ASYNC PIPETO CANCELLATION

December 30, 2025

Migrating from CRUD to CQRS and Event-Sourcing with Akka.Persistence

AKKA.NET AKKA.PERSISTENCE CQRS

// AUTHOR

Bartosz Sypytkowski

CEO & Co-Founder, Petabridge

Creator of Akka.NET. Building distributed systems infrastructure for .NET since 2015. Writes about OSS business models, distributed architecture, and the intersection of AI and systems programming.

twitter.com/Horusiath →

Shares

// SUBSCRIBE

Enjoyed this post? Subscribe to our newsletter for more insights on distributed systems, Akka.NET, and .NET + AI.

First name

you@company.com

Subscribe

Read more about: Akka.NET Business Case Studies Engineering NBench Product Videos

REF: COMMENTS

// COMMENTS

Sign in

Add a comment

Powered by Comenta Shares

REF: NEWSLETTER

// STAY_CONNECTED

The Petabridge Newsletter

When we publish, it's worth reading. Distributed systems architecture, Akka.NET patterns, .NET + AI development, and performance — from the framework's creators.

First name

you@company.com

Subscribe

10,000+ subscribers · 1–2x per month · Unsubscribe anytime

// PETABRIDGE

PLATFORM

Akka.NET

SERVICES

Design-Time Support

Akka.Cluster

Akka.Streams

Akka.Persistence

Phobos

Petabridge.Cmd

Production Support

Architecture Reviews

Training

Consulting

Shares

RESOURCES

Bootcamp

Documentation

Blog

Newsletter

CONNECT

GitHub

Twitter / X

YouTube

About Us

Contact Us

© 2015–2026 Petabridge, LLC · Conroe, TX

// building distributed systems since